

VANADIS 3D v2026.3.0

Technical, mathematical and functional description

Directional Residual stabilization, nonlinear $P(S)$, $Q(t)$, OpenMP and CUDA Element-by-Element architecture

Author	Marek Chodorski
Solver core	VER. 1.08_3
Release	v2026.3.0
Source snapshot	7 September 2026

Source-derived documentation. Mathematical statements below are reconstructed from the supplied implementation, not from an external model specification.

Executive summary

Vanadis 3D is a three-dimensional dynamic Eulerian finite-element model for atmospheric pollutant transport. The supplied v2026.3.0 solver core uses trilinear HEX8 elements, Directional Residual (DR) test-function stabilization, an implicit $\theta=2/3$ transient scheme, a nonlinear concentration-dependent reaction coefficient solved by Picard iteration, and a generalized source function $Q(t)$. The linear algebra is kept in Element-by-Element (EBE) form: 64 coefficients are stored per element in the shared array `bb` and no conventional global sparse matrix is formed.

- CPU: element construction and right-hand-side assembly are parallelized with OpenMP.
- GPU: a Fortran-callable CUDA DBCG solver performs EBE $A*x$ and A^T*x directly from `bb` and `ISS`.
- Two CUDA EBE strategies are present: atomic accumulation and an 8-color HEX8 variant that removes atomics within each color.
- Current boundary setup: homogeneous Dirichlet on faces 1, 2, 4, 5 and 6; Robin/deposition on face 3 (the $x=0$ ground plane in the supplied mesh).
- Internal concentration units are kg/m^3 ; output T files are written in mg/m^3 .
- The current embedded test mesh has 2800 nodes and 2166 HEX8 elements, $dt=100$ s and final time 2000 s.

Document structure

1. Source basis and scope
 2. Functional architecture
 3. Physical model and units
 4. Mesh and HEX8 geometry
 5. Finite-element formulation
 6. Directional Residual stabilization
 7. Time-dependent source $Q(t)$
 8. Nonlinear reaction $P(S)$
 9. Time integration and Picard iteration
 10. Boundary conditions
 11. Element-by-Element storage and linear algebra
 12. OpenMP implementation and memory
 13. CPU DBCG solver
 14. CUDA atomic EBE solver
 15. CUDA graph coloring
 16. GPU/CPU memory model
 17. Diagnostics, mass and output
 18. Current v2026.3.0 snapshot
 19. Limitations and code-truth notes
 20. How to present Vanadis/DR
- Appendices: routine maps and source integrity

1. Source basis and scope

This document was reconstructed directly from the four primary Vanadis source files supplied for the v2026.3.0 snapshot and, in this verification pass, the two auxiliary CUDA headers book.h and lock.h. It describes what the code actually does. Where comments conflict with executable statements, the executable statement is treated as authoritative.

File	Lines	Role	SHA-256
VANADIS(20260907-085706).FOR	2874	Fortran solver core	01a0114d020963e3ad3d53d8e8b3dbe4ad6fe5167714eb13b873ac0a2dc6cb02
cudatest(3).cu	602	CUDA DBCG / atomic EBE	d16dc1a4bc5098d3a6b8578f78c2a0cce229cdabf7e7325e276cf17b07bd1193
cudatest_gtx1060_launch_optimized(3).cu	602	GTX 1060 launch-optimized CUDA DBCG / atomic EBE	d16dc1a4bc5098d3a6b8578f78c2a0cce229cdabf7e7325e276cf17b07bd1193
cudatest_coloring_gtx1060_optimized(2).cu	803	GTX 1060 launch-optimized CUDA DBCG / 8-color EBE	52ed8c716c2de0bb836698bdcf6070fd74d696e139ebf474244b413b4a2578e
book.h	217	Auxiliary CUDA/NVIDIA sample utilities and error helpers	27fb79dbc14c8325a9a7297b71eea5d75ceb63b91ab8e13cbbf0180973e63d36
lock.h	41	Auxiliary CUDA device spinlock used by scalar reductions	843cd88f4735aedd3513a8db65d8c79d12783609dda41dad8ac8f15ed86d25a3

cudatest(3).cu and cudatest_gtx1060_launch_optimized(3).cu are byte-for-byte identical in the supplied snapshot. The coloring source is a distinct alternative implementation.

The CUDA sources include book.h and lock.h. Both headers were supplied for this verification pass. Static inspection confirms that book.h provides CUDA error-handling and general NVIDIA sample utilities, while lock.h implements the Lock object as a device-side spinlock using atomicCAS for acquisition and atomicExch for release. In the Vanadis CUDA solver, Lock is used for final accumulation of scalar reduction results; EBE MATVEC/MTTVEC synchronization is handled separately by atomicAdd or by graph coloring.

Primary Fortran locations: program 28-103; element formulation 512-703; time/Picard/OpenMP 704-1060; active DR 2049-2206.

2. Functional architecture

The solver is organized as a local finite-element pipeline. The only globally assembled objects required by the iterative solution are vectors such as the right-hand side and the diagonal preconditioner. The system matrix itself remains as a concatenation of 8x8 element blocks.

```

vanadis.cfg -> mesh/connectivity -> boundary face codes -> time step n+1
              -> Picard iteration k
              -> OpenMP: element DR integration -> bb + RHS + diag
              -> homogeneous Dirichlet mask
              -> CPU or CUDA DBCG (EBE A*x and A^T*x)
              -> nonlinear convergence check
              -> T_ttttt.TXT + mass diagnostic
after final time -> X/Y/Z plane files

```

The nonlinear P(S) layer is outside the linear CUDA solver: each Picard iteration rebuilds the element operator on the CPU, and the GPU receives the resulting linearized bb, RHS and diagonal.

3. Physical model and units

The element terms correspond to the following advection-diffusion-reaction equation written component-wise, without vector differential operators:

$$\begin{aligned}
 & dS/dt + V_1*dS/dx + V_2*dS/dy + V_3*dS/dz \\
 & - d/dx(K_x*dS/dx) - d/dy(K_y*dS/dy) - d/dz(K_z*dS/dz) \\
 & + P(S)*S = q_v(t)
 \end{aligned}$$

S is pollutant concentration. q_v is the volumetric source intensity. In the current source routine, the configured total source $Q(t)$ [kg/s] is divided by the source-element volume to obtain q_v .

Quantity	Unit	Meaning
S	kg/m ³	Nodal concentration; written as mg/m ³ after multiplication by 1e6.
t	s	Time.
x,y,z	m	Global coordinates.
V1,V2,V3	m/s	Advection velocity components.
Kx,Ky,Kz	m ² /s	Directional diffusion coefficients; current atmosfera sets all three equal to K_cfg.
P, P0	1/s	Reaction/decay coefficient.
Q	kg/s	Total source strength assigned to a source element.
q_v	kg/(m ³ s)	Volumetric source after division by element volume.
alpha	m/s	Robin/deposition transfer coefficient.
qs	kg/(m ² s)	Boundary flux term by dimensional consistency; current code sets qs=0.

The current public configuration interface exposes one scalar K. Internally, lambda(1:3) and the DR calculation support a diagonal directional diffusion tensor, but atmosfera currently assigns Kx=Ky=Kz=K_cfg. V1-V3 are likewise taken directly from the configuration and are uniform in the supplied implementation.

4. Mesh and HEX8 geometry

The supplied executable snapshot uses a structured but strongly nonuniform Cartesian topology with compile-time dimensions nax=7, nay=20, naz=20. This gives 2800 nodes and 2166 trilinear HEX8 elements.

Axis	Coordinates [m]
x	0, 20, 35, 85, 135, 230, 420
y	-4000, -1500, -900, -700, -500, -400, -270, -180, -90, -25, 25, 90, 180, 270, 400, 500, 700, 900, 1500, 4000
z	same as y

For the supplied mesh, local xi corresponds to global x, eta to global y and zeta to global z. Face 3 is xi=-1 and therefore the x=0 plane; the code labels this as the lower/ground face. The face map is:

Face	Natural plane	Local nodes	Current BC
1	zeta=-1	1,2,3,4	Dirichlet
2	zeta=+1	5,6,7,8	Dirichlet
3	xi=-1	1,4,8,5	Robin / ground
4	xi=+1	2,3,7,6	Dirichlet
5	eta=+1	4,3,7,8	Dirichlet
6	eta=-1	1,2,6,5	Dirichlet

Volume integrals use 2x2x2 Gauss points at $\pm 1/\sqrt{3}$ with unit weights. The Robin face uses a 2x2 surface quadrature. The active DR routine explicitly forms the 3x3 Jacobian and its inverse at every Gauss point.

A valid nonzero Jacobian is an implicit mesh requirement. The active DR routine divides directly by detJ and does not contain an explicit singular/inverted-element guard.

5. Finite-element formulation

The concentration is interpolated from the eight nodal values with standard trilinear HEX8 shape functions N_j :

$$S(x, y, z, t) = \sum(j=1..8) N_j(x, y, z) * S_j(t)$$

Vanadis does not use N_i as the scalar test function in the advection-dominated residual terms. The active DR test value W_i is used in the time/mass, advection, reaction, source and Robin integrals. For the diffusion gradient, however, the code explicitly sets $dW_i/dx=dN_i/dx$, $dW_i/dy=dN_i/dy$ and $dW_i/dz=dN_i/dz$.

At element level the code can be summarized by $C_e dS_e/dt + H_e S_e = F_e$, where H_e is the sum of diffusion, advection, reaction and Robin terms.

Term	Implemented component form
Mass/time	$C_{ij} = \text{Integral_element} [W_i * N_j] dV$
Diffusion	$D_{ij} = \text{Integral_element} [K_x dN_i/dx dN_j/dx + K_y dN_i/dy dN_j/dy + K_z dN_i/dz dN_j/dz] dV$
Advection	$A_{ij} = \text{Integral_element} [W_i (V1 dN_j/dx + V2 dN_j/dy + V3 dN_j/dz)] dV$
Reaction	$R_{ij} = \text{Integral_element} [W_i P_e N_j] dV$
Robin matrix	$B_{ij} = \text{Integral_ground} [\alpha W_i N_j] dA$
Volume source	$F_i(\text{vol}) = \text{Integral_element} [W_i q_v] dV$
Boundary source	$F_i(\text{boundary}) = - \text{Integral_ground} [W_i q_s] dA$
Element operator	$H_e = D_e + A_e + R_e + B_e$

The matrix called cma_n in the source is the DR-weighted mass/test matrix C_e without P . It is reused both for the transient term and to reconstruct the historical reaction contribution in the nonlinear time scheme.

6. Directional Residual (DR) stabilization

The active DR implementation is `gauss_point_hex8DR`. It constructs a streamline-aligned stabilization parameter from the physical velocity, the local element geometry and the diffusion coefficient seen in the flow direction. The formulation is evaluated separately at every Gauss point.

6.1 Flow direction and stream length

```
Vmag = sqrt(V1^2 + V2^2 + V3^2)
ex = V1/Vmag, ey = V2/Vmag, ez = V3/Vmag
```

The unit flow direction is mapped through the inverse Jacobian to the natural-coordinate space:

```
g_xi = invJ11*ex + invJ12*ey + invJ13*ez
g_eta = invJ21*ex + invJ22*ey + invJ23*ez
g_zeta = invJ31*ex + invJ32*ey + invJ33*ez
gnorm = sqrt(g_xi^2 + g_eta^2 + g_zeta^2)
h_stream = 2/gnorm
```

This makes h_{stream} dependent on both element geometry and flow direction. It is therefore not simply one of the Cartesian element edge lengths.

6.2 Directional diffusion and Peclet number

```
K_stream = Kx*ex^2 + Ky*ey^2 + Kz*ez^2
Pe = Vmag*h_stream / (2*K_stream)
```

Although the current `atmosfera` routine sets $K_x=K_y=K_z$, the active DR calculation is already written for diagonal anisotropic diffusion. The code uses the diffusion projected along the streamline direction, not a simple arithmetic average.

6.3 Stable evaluation of $\beta(Pe)$ and τ_{DR}

```
for Pe < 1e-3: beta = Pe/3 - Pe^3/45 + 2*Pe^5/945
for 1e-3 <= Pe <= 50: beta = 1/tanh(Pe) - 1/Pe
for Pe > 50: beta = 1 - 1/Pe
tau_DR = 0.5*h_stream/Vmag * beta
```

The series avoids cancellation at small Pe ; the large- Pe branch avoids an unnecessary hyperbolic evaluation. τ_{DR} is set to zero if $Pe \leq 1e-12$, if $Vmag \leq 1e-14$, or indirectly when $K_{\text{stream}} \leq 1e-14$ because Pe is then set to zero.

6.4 DR test function

$$W_i = N_i + \tau_{DR} * (V1 * dN_i/dx + V2 * dN_i/dy + V3 * dN_i/dz)$$

The modification is aligned with the local flow direction. It changes the test weighting without adding an explicit isotropic artificial diffusion coefficient. In the diffusion term the code deliberately keeps the physical-gradient part standard: $dW/dx=dN/dx$, $dW/dy=dN/dy$, $dW/dz=dN/dz$.

Two historical routines, `shape_hex8dr_base` and `gauss_point_hex8DR_base`, retain the older coordinate-wise DR construction. They are not called by the active element path. Their presence documents the evolution of the stabilization but should not be described as the v2026.3.0 active algorithm.

Code: shape_hex8DR 1879-1958; active gauss_point_hex8DR 2049-2206; historical base 1744-1878 and 1959-2048.

7. Time-dependent source Q(t)

`source_term` is the single routine that defines the temporal source profile. The current executable statement is:

```
if (element == 1083 and time >= 0) Q = Q_cfg; otherwise Q = 0
```

Therefore, in the supplied test profile the source is already active at $t=0$ and remains constant thereafter. Comments that state $Q(0)=0$ are inconsistent with the executable ≥ 0 condition and are documentation errors.

The time integrator calls the source twice for every element: once at the new time and once at the previous time. This is the architectural reason the solver can represent a genuinely time-varying $Q(t)$ after editing `source_term`.

```
q_v(n+1) = Q(t_n+1)/V_element
q_v(n)   = Q(t_n)/V_element
```

The element volume used for this conversion is obtained by summing `detJ` at the eight 2x2x2 Gauss points. The source is then integrated with the DR test value W_i .

8. Nonlinear reaction/decay coefficient P(S)

The current nonlinear coefficient is element-wise and is based on the arithmetic mean of the eight nodal concentrations of the element. During Picard iteration, one mean is formed from the current nonlinear iterate and a second from the frozen previous time level.

```
Sbar_e = (S1 + S2 + ... + S8)/8
S_ref = 5e-6 kg/m3 = 5 mg/m3
K2 = P0/S_ref
P_e = P0 + K2*abs(Sbar_e)
```

At $Sbar_e=5 \text{ mg/m}^3$ the coefficient is $2*P_0$. The absolute value keeps P_e positive even if a small negative numerical undershoot appears. For physically nonnegative concentration, the reaction loss corresponds to $P_0*S + K_2*S^2$. For negative numerical S , the exact implemented expression is $P(abs(Sbar_e))*S$, not $P_0*S+K_2*S^2$.

P_e is not evaluated at Gauss points; it is one scalar per element per Picard iteration. This distinction should be stated explicitly when presenting the nonlinear formulation.

9. Time integration and Picard iteration

The implemented one-step weighted implicit method corresponds to $\theta=2/3$. With C denoting the DR-weighted mass matrix, H the spatial operator and F the load vector, the code implements:

$$C * (S_{n+1} - S_n) / dt + (2/3) * H_{n+1} * S_{n+1} + (1/3) * H_n * S_n = (2/3) * F_{n+1} + (1/3) * F_n$$

After multiplication by 3, the actual element system stored in `bb` has the form:

$$[2 * H_{n+1} + 3 * C / dt] * S_{n+1} = [3 * C / dt - H_n] * S_n + 2 * F_{n+1} + F_n$$

For nonlinear $P(S)$, H_{n+1} is linearized with P from the current Picard iterate, while the historical H_n must contain $P(S_n)$. The code reconstructs the historical reaction part using the difference $(P_n - P_k) * C$ before forming the right-hand side.

9.1 Picard loop

1. Freeze S_n and initialize the nonlinear iterate $S_k=S_n$.
2. For every element compute $Sbar_e(S_k)$, P_k and separately P_n from S_n .
3. Rebuild the complete element operator and loads.
4. Solve the linearized nonsymmetric system with DBCG.
5. Compute the relative change between the new solution and the previous Picard iterate.
6. If the nonlinear tolerance is not met, set S_k to the new solution and repeat.

```
err_NL = sqrt( sum_i (S_new(i)-S_old(i))^2 / sum_i S_new(i)^2 )
NL tolerance = 1e-8, maximum Picard iterations = 20
```

If the denominator is $\leq 1e-30$, the code switches to the absolute L2 change. A failure to reach the Picard tolerance produces a warning but does not stop the time integration.

10. Boundary conditions

10.1 Homogeneous Dirichlet faces

Faces 1, 2, 4, 5 and 6 are assigned $S=0$. Face 3 is intentionally excluded because it carries the Robin/deposition condition.

The current active implementation does not modify each element row and column. Instead, `build_bc_nodes` creates a global integer mask. Before the solver, $RHS=0$, $diag=1$ and the solution guess is forced to zero at masked nodes. Every active $A*x$ and A^T*x operation zeros Dirichlet inputs and outputs.

`WB_1` and a local element-level Dirichlet block are retained as historical/reference code but are commented out in the active time-discrete assembly.

10.2 Robin/deposition ground boundary

On face 3, the weak formulation receives $+\alpha*W_i*N_j$ on the element matrix and $-q_s*W_i$ on the load. With the outward-normal sign convention implied by the diffusion weak form, this corresponds to:

$$K_x*dS/dx*n_x + K_y*dS/dy*n_y + K_z*dS/dz*n_z = -\alpha*S + q_s$$

The current `lic_alfa` routine sets $\alpha=\alpha_{cfg}$ and $q_s=0$. Thus the implemented ground term is a pure concentration-proportional loss/deposition term. Surface integration uses 2x2 Gauss quadrature and the DR test value W_i .

11. Element-by-Element storage and linear algebra

The defining software architecture is that Vanadis never assembles a conventional global sparse coefficient matrix. For each element, the final 8x8 time-discrete matrix is copied to 64 consecutive doubles in `bb`. Connectivity is stored separately in `ISS`.

```
bb size = 64 * N_elements doubles = 512 * N_elements bytes
ISS size = 8 * N_elements integers = 32 * N_elements bytes (for 4-byte integers)
```

$A*x$ is evaluated by looping over elements, reading the 8 nodal input values and accumulating the 8x8 local contribution into global output-vector entries. A^T*x uses the transposed local block. The global RHS and Jacobi diagonal are vectors; the matrix remains EBE.

This organization is also the CPU/GPU interface: Fortran passes `bb`, `ISS`, `RHS`, `DIAG` and the Dirichlet mask directly to the CUDA solver. No CSR/COO conversion is performed.

12. OpenMP implementation and memory behavior

The dominant element loop in `przest3d` is an OpenMP parallel do with default(shared). `bb` is not private, therefore there is one shared `bb` array in CPU memory. Each iteration writes only the 64-value block belonging to its own element.

Element-local 8x8 work matrices and local geometry COMMON blocks are private/threadprivate. This avoids races while keeping the largest element-matrix store shared.

Important memory nuance: `fmv3` and `diag` are full node-length arrays used in OpenMP array reductions. OpenMP reduction semantics require private reduction storage for each participating thread. Consequently `bb` itself is not multiplied by the thread count, but RAM is not strictly thread-count independent.

```
Approximate OpenMP reduction storage per thread = 2 * N_nodes * 8 bytes = 16*N_nodes
bytes
plus small element-local/threadprivate buffers (only a few kB per thread)
```

For the embedded 2800-node test, these two reduction arrays are only about 43.8 KiB per thread, while `bb` is about 1.06 MiB. For very large meshes and very high thread counts, the reduction copies can become a material part of RAM usage even though `bb` remains single-copy.

13. CPU DBCG solver

The Fortran source contains `dbcg_simple`, a preconditioned BiConjugate Gradient implementation for a nonsymmetric system. The file header explicitly references the SLATEC DBCG/SLAP routine lineage. The active v2026.3.0 call in `przest3d` is `kernel_dbcg_simple`, i.e. the external CUDA implementation; the CPU routine remains available as a reference/fallback after changing the call.

```
M = diag(A)
z = r / diag,    zz = rr / diag
linear error = ||r||2 / ||b||2
configured tolerance = 1e-8, maximum iterations = 1000
```

The operator is generally nonsymmetric because of advection and DR test weighting, hence BiCG uses both A^*x and $A^T x$. The preconditioner is a lightweight Jacobi diagonal built during element assembly.

The implementation does not contain explicit breakdown checks for zero/near-zero BiCG scalar denominators or zero diagonal entries. Correct operation therefore assumes a numerically admissible assembled system and preconditioner.

14. CUDA atomic EBE solver

`cudaest(3).cu` and `cudaest_gtx1060_launch_optimized(3).cu` implement the same active atomic EBE solver in the supplied snapshot. The host entry point is extern "C" `kernel_dbcg_simple_`, matching the conventional trailing-underscore Fortran symbol.

The active `MATVEC_C_cu` and `MTTVEC_C_cu` assign elements to CUDA threads using grid-stride loops. Each processed HEX8 element contributes 64 FP64 multiply-add terms. Because neighboring elements share nodes, output accumulation uses native double `atomicAdd`.

A custom `atomicAddD` based on `atomicCAS` is defined but is not called by the active matrix-vector kernels. The active native double `atomicAdd` is consistent with the intended Pascal/GTX 1060 compute capability 6.1 target.

14.1 Launch policy

Kernel class	Block size	Grid cap
Vector kernels	256 threads/block	up to 8 blocks/SM
MATVEC/MTTVEC	256 threads/block	up to 4 blocks/SM
Reductions	512 threads/block	up to 4 blocks/SM

The SM count is detected at runtime, so the launch configuration adapts to the active Pascal device rather than hard-coding a GTX 1060 SM count.

14.2 GPU BiCG sequence

- Copy `X`, `B`, `bb`, `ISS`, `DIAG` and the Dirichlet mask to device memory.
- Apply the boundary mask to `X`, evaluate A^*X , mask the result and form $r=b-Ax$.
- Apply the Jacobi preconditioner to `r` and `rr`.
- Compute $\|b\|$ and $\|r\|$ with GPU reductions.
- Iterate BiCG: dot products, search-vector updates, A^*p , $A^T pp$, solution/residual updates, preconditioning and residual norm.
- Copy `X` back to Fortran and free all device allocations.

The two norm calculations at initialization and the two Jacobi solves inside each BiCG iteration are launched in two CUDA streams and synchronized before use. Scalar reductions finish by accumulating block results through the Lock object defined in lock.h.

15. CUDA 8-color graph coloring

The coloring variant exploits the regular HEX8 topology. An element with logical indices (i,j,k) receives:

```
color = (i mod 2) + 2*(j mod 2) + 4*(k mod 2)    -> colors 0..7
```

Elements of the same color do not share nodes. Therefore MATVEC_color and MTTVEC_color can read the eight input values, accumulate the local 8x8 contribution in registers/local variables and perform one ordinary store per output node without atomicAdd.

The host builds compact element lists and offsets for all eight colors, copies a single COLOR_ELEMENTS list to the GPU and launches the colors sequentially in the default stream. Sequential color launches are necessary because different colors can share nodes and must accumulate into the same output vector in a defined order.

The cost is eight kernel launches per $A \cdot x$ or $A^T \cdot x$ plus one integer element-list array on the GPU. The benefit is removal of atomics within each colored launch. Which variant is faster depends on mesh size, GPU and atomic contention; the source provides both strategies rather than assuming one is universally superior.

16. CPU/GPU memory model

The EBE design makes the dominant matrix storage proportional to the number of elements, not to the number of OpenMP or CUDA threads. CPU RAM and GPU VRAM are physically separate memory spaces; the CUDA solve therefore contains one host bb and one device BB_cu, but no per-thread or per-block matrix copies.

16.1 Dominant storage formulas

```
CPU bb:          512*N_elements bytes
CPU ISS:         32*N_elements bytes
OpenMP reductions: about 16*N_nodes bytes per thread for fmv3+diag
```

```
GPU atomic core (approx.):
BB_cu + ISS_cu + 9 double node vectors + Dirichlet mask
= 544*N_elements + 76*N_nodes bytes
```

```
GPU coloring adds approximately 4*N_elements bytes for COLOR_ELEMENTS_cu
```

The CUDA host wrapper additionally allocates six N-length CPU arrays P, PP, R, RR, Z and ZZ with calloc. In the supplied active code these host arrays are not copied to the device and are not otherwise used before being freed; they appear to be legacy scratch allocations and are a cleanup opportunity.

Another performance-relevant fact visible in the source is that device arrays are allocated, bb/ISS/DIAG are copied, and all arrays are freed inside every kernel_dbcg_simple_ call. Because Picard invokes a new linear solve for each nonlinear iteration, the current design also repeats allocation and host-device transfer overhead for each Picard iteration. Persistent GPU buffers could reduce this overhead, but they are not part of the supplied v2026.3.0 implementation.

17. Diagnostics, mass and output

- At every time step the program prints the nonlinear P(S) minimum/maximum, assembly time, solver time, linear iteration result and Picard relative change.
- It reconstructs diagonal/off-diagonal row sums from EBE blocks to report strict diagonal dominance and a dominance factor $\max(\text{offdiag_abs_sum}/\text{diag_abs})$.
- At t=1000 s it writes heatmap.ppm, a 300x300 image of the magnitude/distribution of matrix coefficients reconstructed from EBE blocks. It is a matrix-structure diagnostic, not a concentration map.
- Total pollutant mass is independently integrated after each time step with standard HEX8 N functions and 2x2x2 Gauss quadrature. Internal S [kg/m3] times volume gives mass in kg.
- Each T_XXXXX.TXT contains X, Y, Z and S in mg/m3. Concentration is converted from kg/m3 by multiplying by 1e6.

- `vanadis_files` reads each 3D T file and produces constant-X, constant-Y and constant-Z plane files using a 100-record text buffer.

18. Current v2026.3.0 source snapshot

Setting	Value
Solver header	VER. 1.08_3, 06.09.2026
nax, nay, naz	7, 20, 20
Nodes	2800
HEX8 elements	2166
Initial concentration	S=0 at t=0
Time step	100 s
Final time	2000 s
Linear tolerance	1e-8
Linear iteration limit	1000
Picard tolerance	1e-8
Picard iteration limit	20
Current source location	element 1083
Current source profile	$Q=Q_cfg$ for $t \geq 0$
P reference concentration	5 mg/m ³
Dirichlet faces	1, 2, 4, 5, 6
Robin/deposition face	3 (x=0 ground)
Robin qs	0
Volume quadrature	2x2x2
Robin quadrature	2x2
Output concentration	mg/m ³

The numerical values Q_cfg , K_cfg , P_cfg , $V1_cfg$, $V2_cfg$, $V3_cfg$ and α_cfg are read from `vanadis.cfg`. That run-time configuration file was not supplied with this documentation set, so this document does not assign values to those run-time parameters.

19. Limitations, assumptions and code-truth notes

- The mesh topology and coordinate arrays are compiled/hard-coded in the supplied source. Changing the grid requires source changes/recompilation unless this layer is generalized.
- The current source location is hard-coded as element 1083. $Q(t)$ is architecturally separated, but the supplied profile is constant for $t \geq 0$.
- Velocity is spatially and temporally uniform in the supplied `atmosfera` routine. The code structure could be extended, but such variability is not active here.
- The input exposes one K and the supplied atmosphere uses isotropic K , although the element/DR routines carry three directional coefficients.
- $P(S)$ is one scalar per element derived from the 8-node mean, not a Gauss-point or nodal nonlinear coefficient.
- The only active Robin boundary is face 3 and qs is currently zero.

Time-history caveat for future Robin forcing: `fmv_n` stores the previous-time volumetric $Q(t_n)$ contribution, but the previous-time Robin load associated with qs is not separately retained. This is consistent with the current code because $qs=0$. If a nonzero or time-dependent qs is introduced, the historical F^n term should be extended accordingly.

- The active Dirichlet treatment is homogeneous only ($S=0$) through masking. Nonzero historical `WB_1` machinery is present but inactive.
- No explicit `detJ` quality/singularity check is performed in the active DR Gauss routine.

- BiCG has no explicit scalar-breakdown/zero-diagonal guards.
- CUDA allocations and host-device matrix/connectivity transfers are repeated for each linear solve.
- The coloring rule assumes the regular structured HEX8 logical indexing used by the current mesh generator.
- The current source contains comments claiming $Q(0)=0$, but the executable condition $\text{time} \geq 0$ makes $Q(0)=Q_{\text{cfg}}$.
- The auxiliary CUDA headers book.h and lock.h were inspected in this verification pass. lock.h confirms the spinlock mechanism used by GPU scalar reductions; these headers are auxiliary support code and do not alter the Vanadis FEM/DR formulation.
- The code contains historical/base routines and old post-processing paths; their presence should not be confused with the active v2026.3.0 path.

20. How to present Vanadis / DR

The following statements are strongly supported by the supplied implementation and form a concise technical presentation of the model:

- Vanadis is a 3D transient Eulerian FEM atmospheric pollutant transport model using trilinear HEX8 elements.
- Directional Residual stabilization modifies the test function along the local flow direction using a Jacobian-based stream length and the diffusion projected onto that direction.
- The active DR parameter uses a safe small-/moderate-/large-Pe evaluation and reduces to the standard test function when $\tau_{\text{DR}}=0$.
- The diffusion gradient is kept standard ($dW=dN$ in the implemented diffusion term), avoiding an explicit artificial isotropic diffusion matrix.
- The complete linear operator is stored Element-by-Element as one 8x8 block per element; no global sparse matrix is formed.
- OpenMP parallelizes CPU element construction, while CUDA can solve the nonsymmetric EBE system by BiCG with diagonal/Jacobi preconditioning.
- CUDA provides both atomic accumulation and 8-color structured-HEX8 matrix-vector kernels.
- v2026.3.0 adds nonlinear concentration-dependent $P(S)$ through Picard iteration and evaluates source loads at both time levels for generalized $Q(t)$.
- The implicit $\theta=2/3$ scheme consistently combines the previous and new time levels for both the operator and the source.
- Homogeneous Dirichlet boundaries are enforced by masking in every EBE A^*x and $A^T x$; the ground face uses a Robin/deposition term.
- The matrix-storage cost is dominated by one bb array on the CPU and one BB_cu array on the GPU during a CUDA solve, rather than by copies per processing thread.

20.1 Claims to avoid or qualify

- Do not describe v2026.3.0 as a full atmospheric chemistry model; the supplied equation is a transport-diffusion-reaction model with a scalar pollutant field.
- Do not claim arbitrary spatially varying meteorology from this source snapshot; V and K are currently uniform inputs.
- Do not state that $P(S)$ is pointwise at Gauss points; it is element-average based.
- Do not state that the current $Q(t)$ test is time-varying; the architecture supports $Q(t)$, but the supplied profile is constant from $t=0$.
- Do not state that OpenMP memory is completely independent of thread count; bb is shared, but the fmv3/diag array reductions create per-thread storage.
- Do not present the coloring variant as universally faster than atomics without benchmark evidence for the target mesh/GPU.

Appendix A. Fortran procedure map

Routine	Lines	Role
program vanadis	28-103	Main program: initialization, configuration, time loop, mass diagnostic and 2D post-processing.
wsp_na_o	104-167	Defines hard-coded nonuniform grid coordinates and total element/node counts.
licz_alfa	168-187	Sets Robin/deposition coefficient alpha from configuration and qs=0.
atmosfera	188-230	Assigns diffusion, base decay, velocity, zero Dirichlet values and calls the source definition.
source_term	231-256	Defines $Q(t)$; current profile: Q_cfg in element 1083 for $t \geq 0$, zero elsewhere.
licz_p	257-281	Computes nonlinear element reaction coefficient P from the mean element concentration.
siatka	282-301	Creates global nodal coordinates from 1D coordinate arrays.
w_el	302-333	Builds HEX8 element-to-node connectivity ISS.
kod_el_dr	334-377	Marks which of the six boundary faces each element belongs to.
par_3d	378-453	Defines HEX8 natural nodal coordinates, Gauss points and local face-node maps.
data3d	454-476	Loads one element connectivity and nodal coordinates into thread-local COMMON data.
m_m	477-496	Forms an 8x8 outer-product matrix from two 8-vectors.
ilo_m_v	497-511	Dense 8x8 matrix times 8-vector multiplication.
el3d	512-703	Builds one element diffusion, advection, reaction, mass/source and Robin contributions using DR.
przest3d	704-1060	Advances one time step: Picard loop, OpenMP assembly, EBE storage, boundary mask, linear solve, output.
matvec	1061-1092	CPU EBE $y=A*x$ without Dirichlet masking (reference/basic variant).
mttvec	1093-1131	CPU EBE $y=A^T*x$ without Dirichlet masking.
sklad_wd	1132-1157	Stores an 8x8 time-discrete element matrix as 64 consecutive values in bb.
dbcg_simple	1158-1237	CPU preconditioned BiConjugate Gradient solver for a nonsymmetric EBE system.
msolve	1238-1248	Jacobi/diagonal preconditioner solve $M*z=r$.
mtsolve	1249-1258	Transpose-preconditioner solve; same diagonal operation.
dnrm2	1259-1269	Euclidean vector norm.
read_cfg	1270-1326	Reads Q , K , $P0$, $V1$, $V2$, $V3$ and alpha from vanadis.cfg and requests user confirmation.
make_output_dir	1327-1336	Creates output directory if absent.
copyright	1337-1351	Prints program banner and author information.
vanadis_files	1352-1569	Buffered post-processing: splits 3D T files into X/Y/Z constant-plane files.
vanadis_files_old	1570-1679	Historical unbuffered/older plane-file post-processing routine.
quad4_3d	1680-1743	Computes a 4-node surface mapping and surface Jacobian for 2x2 Robin quadrature.
shape_hex8dr_base	1744-1878	Historical/base DR shape/test-function implementation retained in source, not used by active gauss routine.

Routine	Lines	Role
shape_hex8DR	1879-1958	Computes active HEX8 N and natural derivatives; W is later overwritten by active DR routine.
gauss_point_hex8DR_base	1959-2048	Historical/base Gauss-point transformation for the older DR formulation.
gauss_point_hex8DR	2049-2206	Active DR kernel: Jacobian, physical derivatives, stream length, directional K, Pe, tau and W.
build_bc_nodes	2207-2243	Builds the homogeneous Dirichlet node mask on faces 1,2,4,5,6.
mark_face_nodes	2244-2329	Maps a local face number to its four HEX8 nodes.
matvec_0	2330-2375	CPU EBE $A \cdot x$ with homogeneous Dirichlet masking of input and output.
mttvec_0	2376-2428	CPU EBE $A^T \cdot x$ with homogeneous Dirichlet masking.
is_diagonally_dominant	2429-2486	Reconstructs row sums from EBE blocks and tests strict diagonal dominance.
ascii_heatmap_ppm	2487-2614	Creates a 300x300 PPM heatmap of absolute EBE matrix structure at $t=1000$ s.
diagonal_dominance_factor	2615-2673	Computes $\max(\text{sum off-diagonal abs} / \text{diagonal abs})$ from EBE data.
integrate_mass_hex8	2674-2744	Integrates total pollutant mass from nodal concentration with 2x2x2 Gauss quadrature.
jacobian_hex8	2745-2783	Computes HEX8 Jacobian and determinant for mass integration.
shape_hex8_only_n	2784-2856	HEX8 N and natural derivatives used by mass integration.
WB_1	2857-2874	Historical/local Dirichlet row/column modification routine; active path uses a solver mask instead.

Appendix B. CUDA kernel map

B.1 Atomic/launch-optimized file

Kernel/function	Line	Role
PIERWSZA_cu	49	Forms initial residual $r=b-Ax$ and shadow residual $rr=r$.
TRZECIA_cu	94	Initializes BiCG search vectors $p=z$ and $pp=zz$.
CZWARTA_cu / CZWARTA2_cu	86/90	Computes scalar BiCG ratios and stores the previous denominator.
PIATA_cu	58	Updates BiCG search vectors.
SZOSTA_cu	76	Updates solution x and residual r .
SIODMA_cu	68	Updates shadow residual rr .
DRUGA_cu	104	Parallel dot product reduction; block sums are accumulated through a device lock.
DNRM2_C_cu	140	Parallel squared L2 norm reduction.
msolve	176	GPU Jacobi preconditioner.
MATVEC_C_cu	184	Atomic EBE $A \cdot x$; one thread iterates over one or more HEX8 elements and accumulates 64 contributions.
MTTVEC_C_cu	207	Atomic EBE $A^T \cdot x$.
apply_dirichlet_border / apply_dirichlet	229/244	Zeros homogeneous Dirichlet entries before and after EBE matrix-vector products.
apply_dirichlet_gstride / branchless / ldg	259/275/287	Alternative Dirichlet kernels retained in source; active calls use border/apply variants.

Kernel/function	Line	Role
kernel_dbcg_simple_	301	Fortran-callable host wrapper: GPU allocation, copies, DBCG loop, result copy-back and cleanup.

B.2 Coloring-specific functions

Function	Lines	Role
generate_hex8_colors	50-74	Assigns one of eight colors by element-index parity: $(i \bmod 2) + 2(j \bmod 2) + 4(k \bmod 2)$.
build_color_element_lists	82-106	Builds compact contiguous element lists and offsets for each of the eight colors.
MATVEC_color	289-325	EBE A^*x for one color; no atomics because same-color elements share no nodes.
MTTVEC_color	327-360	EBE $A^T x$ for one color without atomics.
kernel_dbcg_simple_	434-803	Builds coloring once per linear solve, launches eight color kernels sequentially in the default stream and runs DBCG.

Appendix C. Source integrity and inspection notes

The Fortran source was syntax-compiled to an object during preparation of this documentation using gfortran with fixed-form, legacy and OpenMP options; the compilation completed successfully. This is an inspection check only, not a full linked CUDA run.

The Fortran source was compiled successfully to an object file with gfortran using fixed-form, legacy and OpenMP options plus extensive warning/checking flags. The CUDA sources were not compiled in this documentation environment because the CUDA compiler/toolkit (nvcc) is not installed. With book.h and lock.h now supplied, their required declarations and the Lock implementation were nevertheless verified by static inspection. The supplied auxiliary headers carry their own NVIDIA copyright/EULA notices and are treated here as external support dependencies rather than part of the Vanadis numerical formulation.

File	SHA-256
VANADIS(20260907-085706).FOR	01a0114d020963e3ad3d53d8e8b3dbe4ad6fe5167714eb13b873ac0a2dc6cb02
cudatest(3).cu	d16dc1a4bc5098d3a6b8578f78c2a0cce229cdabf7325e276cf17b07bd1193
cudatest_gtx1060_launch_optimized(3).cu	d16dc1a4bc5098d3a6b8578f78c2a0cce229cdabf7325e276cf17b07bd1193
cudatest_coloring_gtx1060_optimized(2).cu	52ed8c716c2de0bb836698bdcf6070df74d696e139ebf474244b413b4a2578e
book.h	27fb79dbc14c8325a9a7297b71eea5d75ceb63b91ab8e13cdbf0180973e63d36
lock.h	843cd88f4735aedef3513a8db65d8c79d12783609dda41dad8ac8f15ed86d25a3

Document status: source-derived technical description for Vanadis 3D v2026.3.0. It is suitable as project documentation and as a basis for technical presentation, but it does not replace validation studies, benchmark reports or a formal numerical-analysis proof.